

Propuesta de proyecto propio: JavaBeasts

Alumno: Marcos Miquel Lisarde

Descripción y justificación del proyecto

JavaBeasts es una aplicación multijugador orientada al entretenimiento digital que permite a varios jugadores participar en combates por turnos utilizando dispositivoPropuesta de proyecto propio:

JavaBeastss móviles conectados a una sesión común. El sistema se basa en un modelo cliente-servidor donde los jugadores interactúan y eligen sus ataques o acciones desde sus dispositivos mientras el estado global del juego se muestra en una pantalla principal.

El proyecto encaja dentro del sector del entretenimiento digital y los videojuegos multijugador, un ámbito que está en crecimiento constante. Según diversos estudios^{[1][2]} del sector, existe una demanda creciente de juegos accesibles, multijugador y con mecánicas simples pero estratégicas que permitan partidas rápidas y compartidas entre usuarios.

JavaBeasts responde a esta demanda proponiendo una experiencia inspirada en juegos como Kahoot o Pokémon Showdown, donde la interacción entre jugadores y la toma de decisiones estratégicas son elementos clave.

A partir de estas necesidades del mercado, se plantea el desarrollo de un sistema cliente-servidor que permita gestionar múltiples usuarios simultáneamente, mantener el estado del juego en tiempo real y ofrecer una experiencia interactiva accesible desde dispositivos móviles, justificando así la viabilidad del proyecto dentro del sector del entretenimiento digital.

[1]: <https://howtomarketagame.com/2024/07/16/what-games-are-selling-q2-2024/>

[2]: <https://howtomarketagame.com/2024/01/25/what-are-the-top-selling-indie-games-of-2023/>

Motivación personal

Siempre me han gustado los videojuegos, y durante un tiempo quise empezar mi propio estudio indie de videojuegos.

Ahora que tengo los conocimientos para ello, quiero desarrollar un proyecto que se acerque a lo que siempre he querido hacer.

He elegido una temática similar a pokémon por el amor que le tengo a esta franquicia, que la he jugado durante años, casi desde que tengo memoria. También creo que será un proceso divertido y motivador.

También me motiva un poco la idea de aprender Spring, ya que Java es mi lenguaje de programación favorito y aprender el framework principal de este me emociona bastante.

Estructura, especificación técnica y evidencias

Cliente

- React Native / Expo para móvil
- JavaFX para el cliente de escritorio

Servidor

- Java como backend completo con SpringBoot
- sockets para tiempo real
- lógica de juego
- gestión de usuarios/equipos
- acceso a base de datos

Persistencia

- MySQL

Otras

- Git / GitHub
- JUnit
- Docker

Ejecución estandarizada

La ejecución se hará por tres partes: el software servidor, la aplicación móvil y la persistencia.

Para el software servidor, se ejecutará un contenedor de docker que contendrá el código.

Para la persistencia, nada más que hará falta levantar el contenedor de docker con el mysql.

Se creará un script .sh y un script .bat para ejecutar ambos contenedores de forma sencilla.

Para la aplicación móvil, se generará un .apk para instalar que, con abrirse, intentará conectar con el servidor

Vector de complejidad elegido

Comunicación en tiempo real con sockets

El sistema utilizará comunicación mediante sockets TCP para permitir la interacción simultánea entre múltiples jugadores conectados al servidor.

Cada cliente móvil se conectará al servidor para enviar las acciones del jugador durante la partida, mientras que el servidor gestionará la lógica del combate y enviará actualizaciones a todos los clientes conectados.

Repositorio

El repositorio se gestionará mediante control de versiones utilizando Git. Se emplearán tres tipos de ramas:

- production: contendrá la versión estable del proyecto lista para su entrega o demostración.
- release: se utilizará como rama intermedia donde se integrarán las funcionalidades completadas antes de ser validadas y fusionadas en production.
- feature/*: se creará una rama por cada funcionalidad, corrección o mejora (feature, fix o refactor), permitiendo un desarrollo modular y organizado.

El flujo de trabajo consistirá en desarrollar cada funcionalidad en una rama `feature/*`, que posteriormente será fusionada en la rama `release`. Una vez validadas las funcionalidades en esta rama, se integrarán en `production`.

Resultados de aprendizaje cubiertos

RAs ya trabajados

- **(Acceso a datos) RA 2: Desarrolla aplicaciones que gestionan información almacenada en bases de datos relacionales identificando y utilizando mecanismos de conexión :**
La aplicación dispondrá de una base de datos relacional (MySQL) que almacenará la información del sistema, incluyendo usuarios, equipos de criaturas, criaturas disponibles y el historial de partidas. El backend desarrollado en Java gestionará la conexión a la base de datos y las operaciones necesarias para consultar, insertar, modificar y eliminar información.
 - **Evidencia 1** : Código en Java implementando la conexión a la base de datos y las operaciones CRUD necesarias para el funcionamiento del sistema, junto con un archivo `.sql` que contenga la estructura de la base de datos y datos de prueba.

- **(Programación de servicios y procesos) RA 3: Programa mecanismos de comunicación en red empleando sockets y analizando el escenario de ejecución :**
El sistema contará con un servidor desarrollado en Java que permitirá la conexión simultánea de múltiples jugadores mediante sockets TCP. Durante las partidas, los clientes enviarán acciones al servidor y recibirán actualizaciones del estado del combate en tiempo real.
 - **Evidencia 2 :** Código fuente del servidor en Java implementando comunicación mediante sockets y gestión concurrente de múltiples clientes mediante hilos.
- **(Programación de servicios y procesos) RA 4: Desarrolla aplicaciones que ofrecen servicios en red, utilizando librerías de clases y aplicando criterios de eficiencia y disponibilidad :**
El backend del sistema se desarrollará utilizando Spring Boot para implementar una API REST que permitirá gestionar usuarios, equipos de criaturas y datos del juego. Esta API será utilizada por los clientes móviles para interactuar con el sistema.
 - **Evidencia 3 :** Código fuente del backend en Java utilizando Spring Boot con endpoints REST para el registro de usuarios, gestión de equipos y consulta de información del sistema.
- **(Programación multimedia y dispositivos móviles) RA 2: Desarrolla aplicaciones para dispositivos móviles analizando y empleando las tecnologías y librerías específicas :**
Se desarrollará una aplicación móvil que permitirá a los jugadores registrarse, gestionar sus equipos de criaturas y participar en combates multijugador. La aplicación se comunicará con el backend mediante peticiones HTTP al ServerSocket y a la API REST.
 - **Evidencia 4 :** Código fuente de la aplicación móvil desarrollada con React Native o Expo que incluya las pantallas de registro, gestión de equipo y selección de acciones durante el combate.

RAs no trabajados

- **(Introducción a la nube pública) RA 1: Comprende los fundamentos de la computación en la nube, sus ventajas frente a sistemas tradicionales, el marco de adopción, los principios de migración y los aspectos clave de facturación :**
Se realizará un estudio teórico sobre la posible migración del sistema JavaBeasts a una infraestructura en la nube, analizando las ventajas frente a una infraestructura local y proponiendo una arquitectura basada en servicios cloud.
 - **Evidencia 5 :** Documento técnico que incluya un diagrama de arquitectura del sistema desplegado en la nube (por ejemplo AWS o equivalente), justificando el uso de servicios como instancias de cómputo para el backend, bases de datos gestionadas y estimación aproximada de costes de infraestructura.

- **(Sostenibilidad) RA 5: Realiza actividades sostenibles minimizando el impacto de las mismas en el medio ambiente :**

Se aplicarán medidas activas de sostenibilidad en el desarrollo del sistema orientadas a la optimización del consumo de recursos. En concreto, se implementarán mecanismos de eficiencia como la reducción de consultas innecesarias a la base de datos, el uso de estructuras de datos optimizadas y la posible incorporación de técnicas de cacheo para minimizar el procesamiento repetitivo. Estas medidas permitirán reducir el uso de CPU, memoria y acceso a red, mejorando la eficiencia energética del sistema.

- **Evidencia 6** : Código del backend donde se implementen estrategias de optimización como reducción de consultas redundantes, uso de cacheo o mejoras en el procesamiento de datos, junto con una breve explicación en la memoria del impacto de estas medidas en la eficiencia del sistema.

Memoria del proyecto actual:

<https://docs.google.com/document/d/1z0JAZYffAWiEA46bMfoXg9iscpfGpez8TRyhjSYnE4Q/edit?usp=sharing>

Contenido del juego:

https://docs.google.com/document/d/1gufne_1Hx2qGak20NkcP3XvXrtA1DY4Smk2MC4L8YpE/edit?usp=sharing